

Theory Questions

Model Context Protocol (MCP) and FastMCP

MCP Fundamentals

- 1. Explain the difference between a tool, a resource, and a prompt in MCP, with a one-line example of when you'd reach for each.**

Tool

Used when the AI needs to perform an action. It takes input, does some work, and returns a result.

Example: A tool can calculate the total marks of a student.

Resource

Used to provide information or data to the AI. It does not perform an action or change anything.

Example: A resource can provide a student's learning history.

Prompt

A predefined instruction or template that helps the user ask the AI to perform a particular task.

Example: A prompt can generate a summary of a student's learning progress.

- 2. A local MCP server usually talks to Claude Desktop over `stdio` instead of a normal HTTP API. What does that mean in practice, and why does it make sense for this use case?**

A local MCP server communicates with Claude Desktop using `stdio` (standard input and output) instead of HTTP.

Claude Desktop starts the MCP server on the same computer and communicates with it directly. The server receives requests through standard input and sends the results back through standard output.

This is useful for local use because:

- there is no need to set up an HTTP server or handle network connections;
- the setup stays simple when both Claude Desktop and the MCP server run on the same computer.

FastMCP Documentation Questions

3. FastMCP supports three transports: `stdio`, `http`, and `sse`. Read the deployment docs and explain when you'd pick `stdio` vs. `http`, and what actually changes in your `mcp.run()` call to switch between them.

- **`stdio`** is mainly used when the MCP server runs locally, such as with Claude Desktop. It is simple and needs no network connection.
- **`HTTP`** is used when the server must be accessed over a network. It is useful when multiple clients need to connect to the same server or when the server is deployed remotely.

By default, `mcp.run()` uses `stdio`:

```
mcp.run()
```

To use HTTP, specify the transport and usually the host and port:

```
mcp.run(transport="http", host="127.0.0.1", port=8000)
```

So the main change is adding `transport="http"` together with the required network settings such as host and port.

Reference: <https://gofastmcp.com/deployment/running-server>

4. FastMCP automatically turns some tool return values into “structured output” and others into plain text. Read the tools docs and explain the difference between what happens when a tool returns a plain string vs. when it returns a dict or a Pydantic model, and why that distinction would matter to a client calling your tool.

- If a tool returns a **plain string**, FastMCP treats the result as text.
- If a tool returns a **dictionary** (dict) or a **Pydantic model**, FastMCP can return the result as structured data, where individual fields can be used separately.
- Structured output is useful when the result contains multiple pieces of information that the client may need to process.

For example, returning a string gives the client only text:

```
"Student scored 85 marks"
```

Returning a dictionary gives separate fields such as `name` and `marks`:

```
{"name": "Gaurav", "marks": 85}
```

This distinction matters because structured data is easier for a client or another program to process, while text is mainly useful when the result is meant to be read by a person.

Reference: <https://github.com/modelcontextprotocol/python-sdk/blob/main/docs/servers/structured-output.md>